

Programação Funcional

Aula 01

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 Introdução
- 2 Cálculos em Haskell
- 3 Hugs: Interpretador de Haskell

Tópicos

- 1 Introdução
- 2 Cálculos em Haskell
- 3 Hugs: Interpretador de Haskell

Tópicos

- 1 Introdução
- 2 Cálculos em Haskell
- 3 Hugs: Interpretador de Haskell

Introdução

Conceitos

- Um programa funcional consiste de uma série de definições de funções.
- A execução de um programa funcional consiste em calcular o valor de uma expressão, usando as funções definidas.
- Linguagem funcional que usaremos: Haskell.
- Os programas escritos em Haskell são geralmente chamados de scripts.

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
    | m >= n = m
    | otherwise = n
```

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

O símbolo `--` faz com que a parte da linha a sua direita seja um comentário.

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int
square x = x * x

allEqual :: Int -> Int -> Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int -> Int -> Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Declara uma nova função especificando o seu tipo. O símbolo "::" pode ser lido como "é do tipo..."

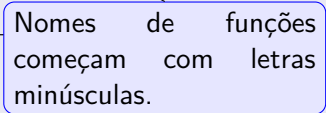
Exemplo

```
-- example.hs
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
    | m >= n = m
    | otherwise = n
```



Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Nomes de tipos começam com letras maiúsculas.

Exemplo

```
-- example.hs
```

```
answer :: Int
```

```
answer = 42
```

```
square :: Int → Int
```

```
square x = x * x
```

Determina que square é
uma função Int para Int.

```
allEqual :: Int → Int → Int → Bool
```

```
allEqual m n p = (m == n) && (n == p)
```

```
maxi :: Int → Int → Int
```

```
maxi m n
```

```
    | m >= n = m
```

```
    | otherwise = n
```

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Equação que determina a função. Define o resultado, $x * x$, da aplicação de `square` sobre `x` ("variável").

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Sintaxe para a aplicação de
uma função f argumentos a, b, c :
 $f\ a\ b\ c$

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Sintaxe para a aplicação de uma função f argumentos a, b, c :
 $f\ a\ b\ c$

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Sintaxe para a aplicação de uma função f argumentos a, b, c :
 $f\ a\ b\ c$

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Sintaxe para a aplicação de
uma função f argumentos a, b, c :
 $f\ a\ b\ c$

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Sintaxe para a aplicação de uma função f argumentos a, b, c :
 $f\ a\ b\ c$

Exemplo

```
-- example.hs
answer :: Int
answer =
square :: Int → Bool
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
    | m >= n = m
    | otherwise = n
```

Declare allEqual como uma função que recebe três objetos Int e retorna Bool.

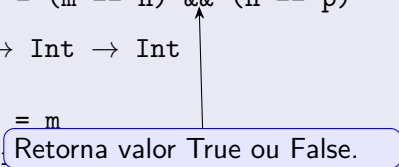
Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherw
```



Retorna valor True ou False.

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Determina que maxi é uma função que recebe 2 objetos Int e retorna Int.

Exemplo

```
-- example.hs
answer :: Int
answer = 42

square :: Int → Int
square x = x * x

allEqual :: Int → Int → Int → Bool
allEqual m n p = (m == n) && (n == p)

maxi :: Int → Int → Int
maxi m n
  | m >= n = m
  | otherwise = n
```

Equação condicional formada por cláusulas. Tudo entre `|` e `=` determina uma guarda. O valor à direita de `=` é retornado se a guarda for verdadeira.

Como cálculos são efetuados em Haskell?

```
allEqual m n p = (m==n) && (n==p)
```

```
allEqual 2 3 3  
= (2==3) && (3==3)  
False && True  
False
```

```
allEqual 5 5 5  
= (5==5) && (5==5)  
True && True  
True
```

Como cálculos são efetuados em Haskell?

Exemplos podem envolver mais de uma função:

```
allEqual (square 3) answer (square 2)
= ((square 3) == answer) && (answer == (square 2))
= ((3*3) == 42) && (42 == (2*2))
= (9 == 42) && (42 == (2*2))
= False && False
= False
```

Como cálculos são efetuados em Haskell?

Exemplo envolvendo equação condicional

```
maxi m n      | m >= n = m  
              | otherwise = n
```

```
maxi 3 1  
?? 3>=1 = True  
= 3
```

```
maxi 3 4  
?? 3>=4 = False  
?? otherwise = True  
= 4
```


Como cálculos são efetuados em Haskell?

Outro exemplo envolvendo equação condicional

```
allEqual (maxi 1 5) 5 (maxi 4 2)  
= ((maxi 1 5) == 5) && (5 == (maxi 4 2))
```

Como cálculos são efetuados em Haskell?

Outro exemplo envolvendo equação condicional

```
allEqual (maxi 1 5) 5 (maxi 4 2)  
= ((maxi 1 5) == 5) && (5 == (maxi 4 2))
```



A ordem de avaliação NÃO importa! Pode-se escolher qualquer uma das expressões e avaliá-la primeiro.

Como cálculos são efetuados em Haskell?

Outro exemplo envolvendo equação condicional

```
allEqual (maxi 1 5) 5 (maxi 4 2)
= ((maxi 1 5) == 5) && (5 == (maxi 4 2))
= ((maxi 1 5 == 5) && (5 == (maxi 4 2)))
?? 1>=5 = False
?? otherwise = True
= (5 == 5) && (5 == (maxi 4 2))
?? 4>=2 = True
= (5==5) && (5 == 4) = True && False
= False
```

Hugs: Interpretador de Haskell

- Ao executar *Hugs*, uma sessão é iniciada.
- O sistema carrega funções pré-definidas (`Prelude.hs`) e passa a esperar comandos:

```
hugs
```

```
Reading file
```

```
"/usr/local/share/hugs/lib/Prelude.hs":
```

```
Hugs session for:
```

```
/usr/local/share/hugs/lib/Prelude.hs
```

```
Type :? for help
```

```
Prelude>
```

Hugs: Interpretador de Haskell

Exemplo de interação - digitando expressões:

```
Prelude> 2+3  
5  
Prelude> (1*6) == (3 'div' 5)  
False  
Prelude> sum [1..10]  
55  
Prelude> reverse "hugs is cool"  
"looc si sguh"  
Prelude>
```

Obs: sublinhado indica entrada digitada pelo usuário.

Hugs: Interpretador de Haskell

- Cada linha digitada é tratada como um *comando*.
- Se for uma expressão, então é tratada como um comando para avaliar a expressão.
- Alguns comandos importantes
 - `:?` imprime a lista de todos os comando;
 - `?q` abandona o interpretador;
 - `:load` carrega definições a partir de um arquivo;

Hugs: Interpretador de Haskell

- Novas definições não podem ser criadas a partir da linha de comando.
- Devem ser carregadas a partir de arquivos.
- Suponha que o script apresentado esteja no arquivo "example.hs":

```
Prelude> :load example.hs  
Reading file "example.hs":
```

```
Hugs session for:
```

```
/usr/local/share/hugs/lib/Prelude.hs
```

```
example.hs
```

```
Main>
```

Hugs: Interpretador de Haskell

- Novas definições não podem ser criadas a partir da linha de comando.
- Devem ser carregadas a partir de arquivos.
- Suponha que o script apresentado esteja no arquivo "example.hs":

```
Prelude> :load example.hs  
Reading file "example.hs".
```

```
Hugs session for:  
/usr/local/share/hugs/  
example.hs  
Main>
```

Um *módulo* é uma coleção de funções, descritas em um arquivo. Se o módulo carregado não tem um nome, o nome *Main* é utilizado.

Hugs: Interpretador de Haskell

- Suponha o arquivo "example.hs" como a seguir:

```
-----  
-- example.hs  
-----  
  
module Exemplo1 where  
answer :: Int  
answer = 42  
...
```

Hugs: Interpretador de Haskell

- Suponha o arquivo "example.hs" como a seguir:

```
-----  
-- example.hs  
-----  
module Exemplo1 where  
answer :: Int  
answer = 42  
...
```

Define o nome do módulo que será carregado.

Hugs: Interpretador de Haskell

- Carregando novamente o arquivo:

```
Prelude> :load example.hs  
Reading file "example.hs":
```

```
Hugs session for:  
/usr/local/share/hugs/lib/Prelude.hs  
example.hs  
Exemplo1> maxi 3 4  
4 Exemplo1>
```

Referências Bibliográficas

Slides:

[1]



Vladimir Oliveira Di Iorio.

Programação funcional - introdução à linguagem funcional haskell.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os
tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos
sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a
bibliografia citada no plano de ensino da disciplina para estudar os
tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.